



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sede Manizales
Sustentacion 2do Parcial, Analisis y Diseno de Algoritmos

Integrantes:
Santiago Marin Giraldo
Tomas Henao Bonilla

1. Planteamiento del problema

Dadas dos cadenas u y v , se busca transformar u en v con el menor numero de operaciones posibles. Las operaciones permitidas son: eliminar un caracter, insertar un caracter o cambiar un caracter. El algoritmo debe devolver la distancia minima y la secuencia de operaciones.

2. Algoritmos

Algoritmo de Fuerza Bruta

La fuerza bruta explora todas las secuencias posibles de operaciones hasta transformar u en v . Se evalúa cada camino y se conserva el de menor costo.

Algorithm 1 Fuerza bruta

```
1: function FUERZA_BRUTA( $u, v$ )
2:   mejor  $\leftarrow \infty$ , mejor_camino  $\leftarrow []$ 
3:   function EXPLORAR( $i, j, costo, camino$ )
4:     if  $i = 0$  y  $j = 0$  then
5:       Actualizar mejor si  $costo$  es menor
6:       return
7:     end if
8:     if  $i = 0$  then
9:       Insertar los  $j$  caracteres restantes de  $v$ 
10:      return
11:    end if
12:    if  $j = 0$  then
13:      Eliminar los  $i$  caracteres restantes de  $u$ 
14:      return
15:    end if
16:    Probar cambiar o mantener el ultimo caracter
17:    Probar eliminar el ultimo caracter de  $u$ 
18:    Probar insertar el ultimo caracter de  $v$ 
19:  end function
20: end function
```

Complejidad

En el peor caso explora un arbol de ramificacion 3, con profundidad aproximada $|u| + |v|$. Por tanto:

$$T(n) = O(3^{|u|+|v|}), \quad S(n) = O(|u| + |v|)$$

Algoritmo Backtracking

El backtracking parte de la fuerza bruta, pero incorpora poda: si el costo actual ya es mayor o igual que la mejor distancia encontrada, se corta ese camino.

Algorithm 2 Backtracking

```
1: function BACKTRACKING( $u, v$ )
2:   mejor  $\leftarrow \infty$ , mejor_camino  $\leftarrow [ ]$ 
3:   function BT( $i, j, costo, camino$ )
4:     if  $costo \geq$  mejor then
5:       return
6:     end if
7:     if  $i = 0$  y  $j = 0$  then
8:       Actualizar mejor y mejor_camino
9:       return
10:    end if
11:    if  $i = 0$  then
12:      Insertar restantes; return
13:    end if
14:    if  $j = 0$  then
15:      Eliminar restantes; return
16:    end if
17:    Probar cambiar o mantener
18:    Probar eliminar
19:    Probar insertar
20:  end function
21: end function
```

Complejidad

En el peor caso mantiene el mismo orden exponencial que fuerza bruta, pero con mejor desempeno en practica gracias a la poda.

$$T(n) = O(3^{|u|+|v|}), \quad S(n) = O(|u| + |v|)$$

Algoritmo Divide y Venceras

Se define un subproblema $resolver(i, j)$ que calcula la distancia minima entre los primeros i caracteres de u y los primeros j caracteres de v . En cada llamada se exploran las tres operaciones posibles y se escoge la de menor costo. La implementacion recursiva devuelve tambien la secuencia de operaciones.

Algorithm 3 Divide y venceras

```
1: function DYV( $u, v$ )
2:   function RESOLVER( $i, j$ )
3:     if  $i = 0$  then
4:       return  $j$ , lista de inserciones
5:     end if
6:     if  $j = 0$  then
7:       return  $i$ , lista de eliminaciones
8:     end if
9:      $(c_1, ops_1) \leftarrow \text{Resolver}(i - 1, j); c_1 \leftarrow c_1 + 1$ 
10:     $(c_2, ops_2) \leftarrow \text{Resolver}(i, j - 1); c_2 \leftarrow c_2 + 1$ 
11:     $(c_3, ops_3) \leftarrow \text{Resolver}(i - 1, j - 1)$ 
12:    if  $u[i - 1] \neq v[j - 1]$  then  $c_3 \leftarrow c_3 + 1$ 
13:    end if
14:    return la opcion con menor costo
15:  end function
16: end function
```

Complejidad

En el peor caos, explora un arbol de llamadas exponencial.

$$T(n) = O(3^{|u|+|v|}), \quad S(n) = O(|u| + |v|)$$

Algoritmo de Programación Dinámica con Tabulación

Se construye una tabla $dp[i][j]$ con la distancia minima entre los primeros i caracteres de u y los primeros j caracteres de v . La recurrencia es:

$$dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + \delta)$$

donde $\delta = 0$ si los caracteres son iguales y $\delta = 1$ si son diferentes. Además se guarda el paso para reconstruir las operaciones.

Algorithm 4 Tabulacion

```
1: function TABULACION( $u, v$ )
2:   Inicializar  $dp$  y  $pasos$ 
3:   Llenar  $dp$  con la recurrencia
4:   Reconstruir operaciones desde  $(|u|, |v|)$  usando  $pasos$ 
5:   return  $dp[|u|][|v|]$ , operaciones
6: end function
```

Complejidad

$$T(n) = O(|u| \cdot |v|), \quad S(n) = O(|u| \cdot |v|)$$

3. Pruebas

Pruebas unitarias y validacion de correctitud

La correctitud se verifico mediante pruebas unitarias comparando la distancia obtenida por cada algoritmo sobre un conjunto de casos deterministas y casos frontera. Se utilizo el siguiente procedimiento:

```
def evaluar_correctitud(algoritmo, casos):
    resultados = []
    for u, v in casos:
        distancias = {}
        for nombre, func in algoritmo.items():
            distancia, operaciones = func(u, v)
            distancias[nombre] = distancia
        resultados.append((u, v, distancias))

    return resultados
```

Los principales casos deterministas evaluados fueron:

Cadena Base	Cadena Objetivo	Distancia Obtenida
vacio	vacio	0
hola	hola	0
gato	gata	1
abba	abcbc	2
mesa	vacio	4
vacio	casa	4

Para todos los casos evaluados, las cuatro implementaciones produjeron la misma distancia minima. Lo cual prueba la correctitud de las implementaciones y su consistencia entre sí.

Verificacion de operaciones

Ademas de comparar la distancia, se revisaron las secuencias de operaciones generadas para transformar una cadena en otra. Se uso el siguiente procedimiento:

```
u, v = "gato", "pato"

for nombre, funcion in algoritmos.items():
    distancia, operaciones = funcion(u, v)
    print(nombre, "->", distancia)
    for operacion in operaciones:
        print("-", operacion)
    print()
```

Para el ejemplo gato $\rightarrow$ pato, todas las implementaciones detectaron que la operacion minima es cambiar g por p, manteniendo los demas caracteres.

Rendimiento experimental

Se realizo una comparacion de tiempos promedio para entradas de tamaños pequeños, usando cadenas aleatorias con alfabeto $\{a, b, c, d\}$. Para cada tamaño n se midio el tiempo de ejecucion de cada algoritmo con dos repeticiones y se reporto el promedio.

```
def medir_tiempo(fn, u, v, repeticiones=2):
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.time()
        fn(u, v)
        fin = time.time()
        tiempos.append((fin - inicio) * 1000)
    return statistics.mean(tiempos)

tamanos = [2, 3, 4, 5]
for n in tamanos:
    u = generar_cadena(n)
    v = generar_cadena(n)
    for nombre, fn in algoritmos.items():
        prom_ms = medir_tiempo(fn, u, v)
```

Los resultados muestran el comportamiento esperado: fuerza bruta y divide y venceras crecen rapidamente, backtracking mejora con la poda pero sigue siendo exponencial, y la tabulacion mantiene un crecimiento mucho mas controlado.

n	Fuerza Bruta (ms)	Backtracking (ms)	DyV (ms)	Tabulacion (ms)
2	0.0218	0.0035	0.0061	0.0039
3	0.0246	0.0075	0.0230	0.0044
4	0.1155	0.0105	0.1096	0.0055
5	0.6119	0.0570	0.5695	0.0069

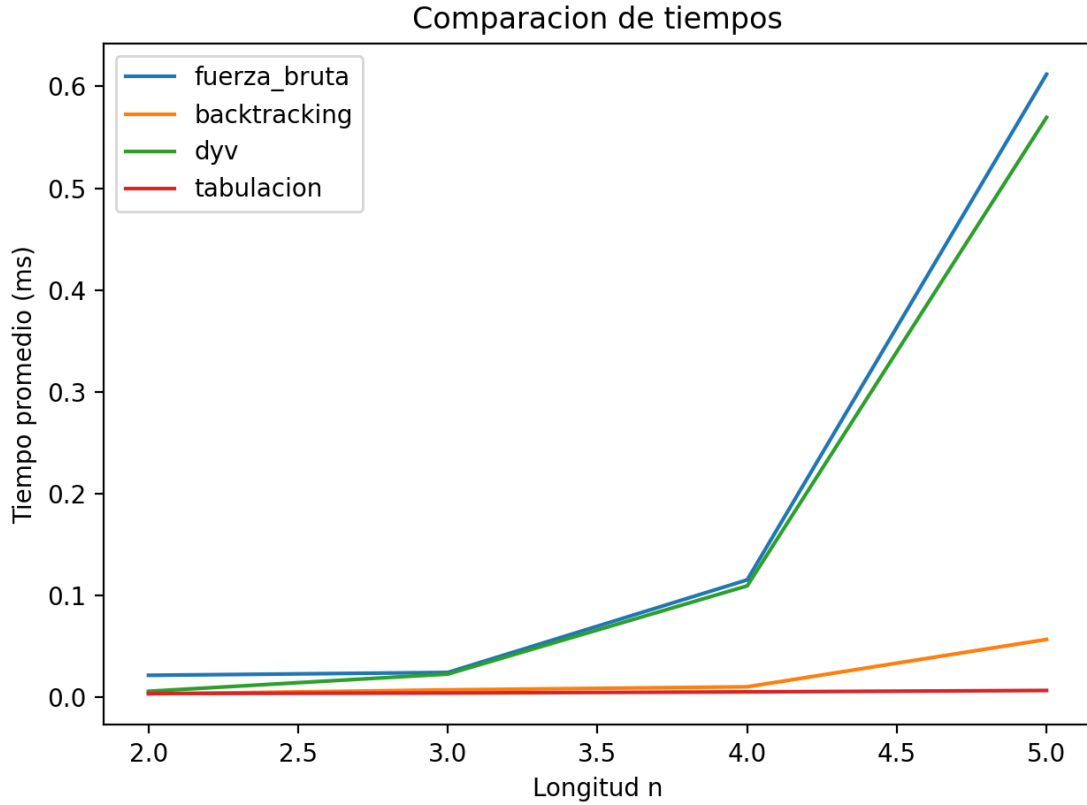


Figura 1: Comparacion de tiempos promedio por tamano de entrada

4. Conclusiones

- La fuerza bruta garantiza optimalidad pero es inviable para entradas medianas.
- El backtracking mejora en practica gracias a la poda, aunque el peor caso sigue siendo exponencial.
- La programacion dinamica con tabulacion ofrece el mejor equilibrio entre costo y tamano de entrada.
- Las pruebas unitarias confirmaron la consistencia de distancias y operaciones.